

CodeCruncher — Containerized Deployment Using Docker

Project Overview

CodeCruncher is a React + TypeScript + Vite-based web application that has been containerized using Docker.

The project demonstrates the complete Docker containerization workflow:

Developer → Dockerfile → Docker Image → Docker Container → Nginx → CodeCruncher Application

The project also uses Docker Compose to run the CodeCruncher application together with a PostgreSQL database.

The Docker architecture consists of:

CodeCruncher Application Container + PostgreSQL Database Container → Docker Network → Persistent Database Volume

The application is built using Node.js and Vite and served in production using Nginx. The Dockerfile uses a multi-stage build where Node.js is used for compiling the application and Nginx is used as the lightweight runtime server.

Project Objectives

The main objectives of this project are:

- Understand Docker and containerization.
- Understand Docker architecture and its major components.
- Create a production-ready Dockerfile.
- Understand multi-stage Docker builds.
- Build and manage Docker images.
- Create and manage Docker containers.
- Run the CodeCruncher application using Docker.
- Serve a React/Vite application using Nginx.
- Configure Nginx for a Single Page Application.
- Use Docker Compose for multi-container deployment.
- Run PostgreSQL using Docker Compose.
- Create and use a Docker bridge network.
- Use Docker service names for container communication.
- Use a persistent Docker volume for PostgreSQL.
- Verify application and database containers.
- Monitor container logs.
- Inspect Docker containers, networks and volumes.

- Troubleshoot common Docker problems.
- Apply Docker security and optimization practices.

Technologies Used

Technology	Purpose
React	Frontend application framework
TypeScript	Application development language
Vite	Frontend build tool
Node.js 20 Alpine	Docker build environment
Nginx 1.27 Alpine	Production web server
Docker	Application containerization
Docker Compose	Multi-container application management
PostgreSQL 16 Alpine	Database
Docker Network	Container-to-container communication
Docker Volume	Persistent database storage
Git	Version control
GitHub	Source code repository

The project uses React, React DOM, Vite, TypeScript and related frontend dependencies defined in `package.json`.

Project Structure

```
CodeCruncher-website/
|
├── Dockerfile
├── compose.yaml
├── nginx.conf
├── .dockerignore
├── .gitignore
├── package.json
├── package-lock.json
├── index.html
├──
├── src/
│   └── ...
├──
├── supabase/
│   └── migrations/
```

```
|
| | Screenshots/
| | | ...
| |
| | vite.config.ts
```

Important Docker Files

File	Purpose
Dockerfile	Defines how the CodeCruncher image is built
compose.yaml	Defines the application and PostgreSQL services
nginx.conf	Configures Nginx to serve the frontend
.dockerignore	Prevents unnecessary files from entering the Docker build context
package.json	Defines frontend dependencies and build scripts
package-lock.json	Locks dependency versions for reproducible installation

The repository contains these Docker-related files along with the application source code.

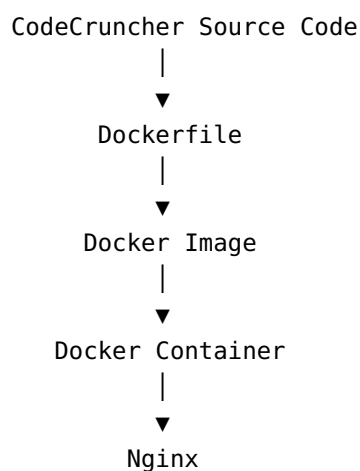
1. Docker Containerization

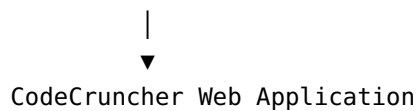
Containerization packages an application together with the files and runtime environment required to execute it.

Docker provides isolated containers that allow applications to run consistently across different environments.

For CodeCruncher, Docker packages the built frontend and Nginx into a portable container.

The basic workflow is:





Virtual Machine vs Docker Container

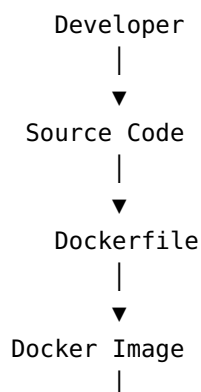
Virtual Machine	Docker Container
Includes a complete guest operating system	Shares the host operating system kernel
Higher resource usage	Lightweight
Slower startup	Fast startup
Requires more storage	Usually requires less storage
Virtualizes hardware	Provides application-level isolation

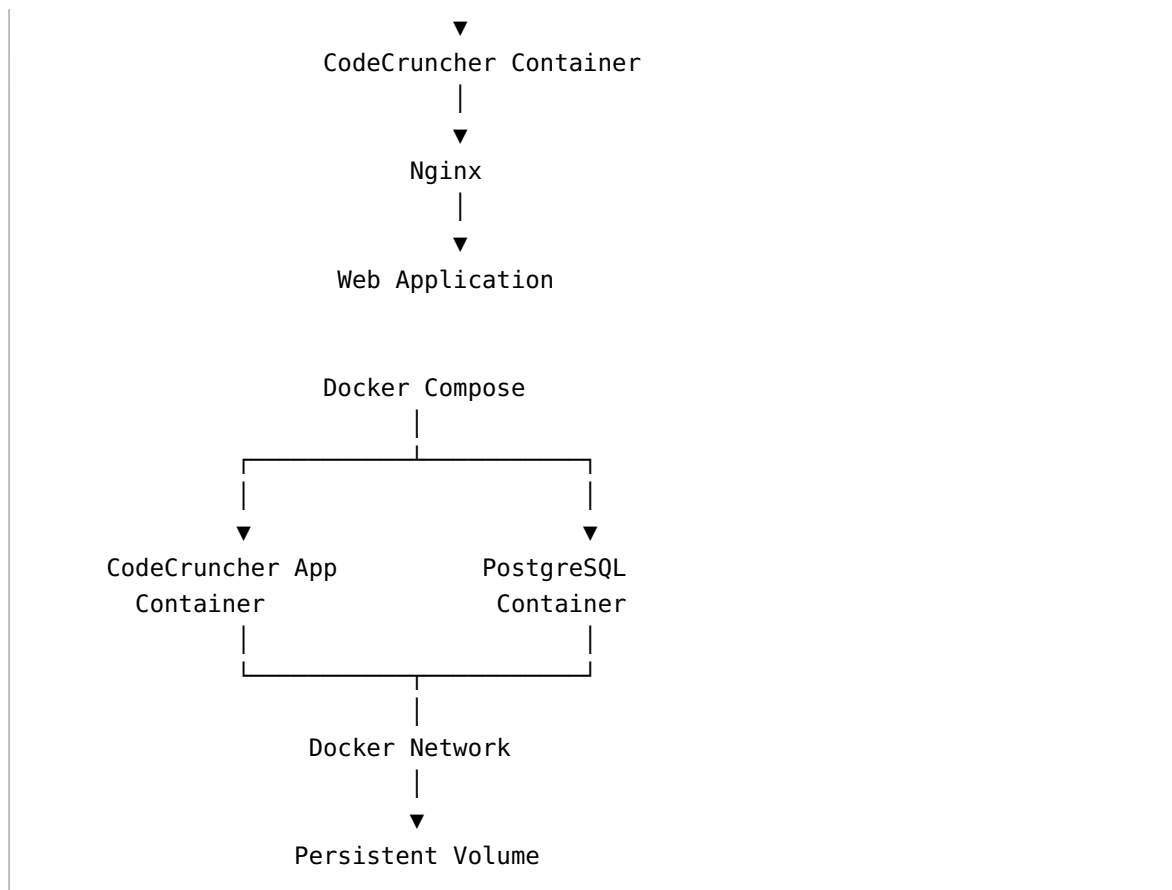
2. Docker Architecture

The major Docker components used in this project are:

- **Docker Client** — command-line interface used to interact with Docker.
- **Docker Engine** — platform responsible for building and running containers.
- **Docker Daemon** — background service responsible for managing Docker objects.
- **Docker Image** — read-only template used to create containers.
- **Docker Container** — running instance of a Docker image.
- **Dockerfile** — instructions used to build a Docker image.
- **Docker Compose** — tool used to define and manage multiple containers.
- **Docker Network** — allows containers to communicate with each other.
- **Docker Volume** — provides persistent storage for container data.
- **Docker Hub** — public registry from which base images can be downloaded.

CodeCruncher Docker Architecture





3. Docker Installation

Docker must be installed on the host machine before the CodeCruncher containers can be created.

Docker installation can be verified using:

```
docker --version
```

Check Docker information:

```
docker info
```

Check running containers:

```
docker ps
```

Check available images:

```
docker images
```

Check Docker Compose:

```
docker compose version
```

Check Docker service:

```
sudo systemctl status docker
```

Docker Service Management

Start Docker:

```
sudo systemctl start docker
```

Stop Docker:

```
sudo systemctl stop docker
```

Restart Docker:

```
sudo systemctl restart docker
```

Enable Docker at system startup:

```
sudo systemctl enable docker
```

Docker Installation Verification

A basic Docker installation can be tested using:

```
docker run hello-world
```

Successful execution confirms that:

- Docker client can communicate with the Docker daemon.
- Docker can download images.

- Docker can create containers.
 - Docker containers can execute successfully.
-

4. CodeCruncher Application

CodeCruncher is a frontend application developed using:

- React
- TypeScript
- Vite

The project uses Vite to build the production frontend.

The build command defined in `package.json` is:

```
npm run build
```

The build process generates the production files inside:

```
dist/
```

These generated files are later copied into the Nginx image.

The project's package configuration defines `npm ci`-compatible dependency installation through the package lock file and the production build through `npm run build`.

5. Dockerfile

The CodeCruncher project uses a **multi-stage Dockerfile**.

```
# Build stage – compile the Vite app
FROM node:20-alpine AS build

WORKDIR /app

COPY package.json package-lock.json* ./

RUN npm ci

COPY . .

RUN npm run build
```

```
# Serve stage – static files via nginx
FROM nginx:1.27-alpine

COPY --from=build /app/dist /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

This Dockerfile is present in the repository and uses Node.js 20 Alpine for the build stage and Nginx 1.27 Alpine for the runtime stage.

Dockerfile Instructions

Instruction	Purpose
FROM	Selects the base image
AS	Gives a name to a build stage
WORKDIR	Sets the working directory
COPY	Copies files into the image
RUN	Executes commands during image creation
EXPOSE	Documents the container port
CMD	Defines the default container startup command

6. Multi-Stage Docker Build

The CodeCruncher Dockerfile uses two stages.

Stage 1 — Build Stage

```
FROM node:20-alpine AS build
```

Node.js is required to install dependencies and compile the React/Vite application.

The working directory is:

```
WORKDIR /app
```

Package files are copied:

```
COPY package.json package-lock.json* ./
```

Dependencies are installed:

```
RUN npm ci
```

Application source code is copied:

```
COPY . .
```

The production application is built:

```
RUN npm run build
```

This generates the production `dist` directory.

Stage 2 — Nginx Runtime

The second stage uses:

```
FROM nginx:1.27-alpine
```

The compiled application is copied from the build stage:

```
COPY --from=build /app/dist /usr/share/nginx/html
```

The custom Nginx configuration is copied:

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

The container exposes:

```
80
```

Nginx is started using:

```
CMD ["nginx", "-g", "daemon off;"]
```

Advantages of Multi-Stage Builds

Multi-stage builds provide several advantages:

- Separate build environment from runtime environment.
- Keep development dependencies out of the runtime image.
- Reduce final image size.
- Improve container security.
- Provide a lightweight production runtime.
- Keep the production container focused only on serving the application.

7. Nginx Configuration

The CodeCruncher project includes:

```
nginx.conf
```

The Nginx server listens on port:

```
80
```

The document root is:

```
/usr/share/nginx/html
```

The configuration includes SPA routing:

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

This allows frontend routes to correctly fall back to `index.html`.

The configuration also enables:

- Browser caching for static assets.
- Security headers.
- Gzip compression.
- Nginx access-log optimization for static assets.

The repository's Nginx configuration explicitly includes SPA fallback, one-year caching for hashed static assets, security headers, and gzip compression.

8. Building the Docker Image

The CodeCruncher image can be built using:

```
docker build -t codecruncher:v1 .
```

Explanation:

```
docker build
```

Builds the image.

```
-t codecruncher:v1
```

Assigns the image name and tag.

```
.
```

Uses the current directory as the Docker build context.

Verify the Image

```
docker images
```

Expected image:

```
codecruncher:v1
```

Inspect the Image

```
docker inspect codecruncher:v1
```

View image history:

```
docker history codecruncher:v1
```

View image size:

```
docker images codecruncher:v1
```

9. Running the Docker Container

The CodeCruncher container can be started using:

```
docker run -d
  --name codecruncher-container
  -p 8080:80
  codecruncher:v1
```

Port Mapping

```
graph TD
    A[Host Port 8080] --> B[Container Port 80]
    B --> C[Nginx]
    C --> D[CodeCruncher Application]
```

The application can then be accessed through:

```
http://localhost:8080
```

The repository's Docker deployment report also records the application using host port `8080` mapped to container port `80`.

10. Container Verification

Check running containers:

```
docker ps
```

Check all containers:

```
docker ps -a
```

Check the CodeCruncher container:

```
docker ps --filter name=codecruncher-container
```

Verify Application Response

Use:

```
curl -I http://localhost:8080
```

A successful deployment should return an HTTP response such as:

```
HTTP/1.1 200 OK
```

11. Docker Container Logs

View application container logs:

```
docker logs codecruncher-container
```

Follow logs in real time:

```
docker logs -f codecruncher-container
```

View the latest 50 lines:

```
docker logs --tail 50 codecruncher-container
```

Logs are useful for identifying:

- Nginx startup problems.

- Configuration errors.
 - Container startup failures.
 - Runtime problems.
-

12. Docker Container Inspection

Inspect the container:

```
docker inspect codecruncher-container
```

Useful information includes:

- Container ID
- Image
- Port mappings
- Network configuration
- Mounts
- Environment variables
- Container state

Check container processes:

```
docker top codecruncher-container
```

Check resource usage:

```
docker stats codecruncher-container
```

13. Container Management Commands

Stop the container:

```
docker stop codecruncher-container
```

Start the container:

```
docker start codecruncher-container
```

Restart the container:

```
docker restart codecruncher-container
```

Remove the container:

```
docker rm codecruncher-container
```

Force remove:

```
docker rm -f codecruncher-container
```

Remove the image:

```
docker rmi codecruncher:v1
```

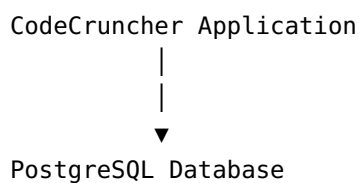
14. Docker Compose

Docker Compose is used to define and manage the CodeCruncher application and PostgreSQL database as a single application stack.

The project contains:

```
compose.yaml
```

The Compose configuration defines two services:



The repository's Compose file defines:

- `codecruncher` application service.
 - `database` PostgreSQL service.
 - `codecruncher-network` bridge network.
 - `codecruncher-db-data` persistent volume.
-

15. Docker Compose Configuration

The application service builds from:

```
build:  
  context: .  
  dockerfile: Dockerfile
```

The application container is named:

```
codecruncher-app
```

Port mapping:

```
ports:  
  - "8080:80"
```

The application also uses:

```
restart: unless-stopped
```

and depends on the database:

```
depends_on:  
  - database
```

PostgreSQL Service

The database uses:

```
image: postgres:16-alpine
```

The database container is:

```
codecruncher-db
```

The configured database is:

```
codecruncher
```

The configured database user is:

```
codecruncher
```

PostgreSQL stores its data at:

```
/var/lib/postgresql/data
```

The Compose configuration maps that directory to a persistent Docker volume.

16. Docker Compose Commands

Build and start all services:

```
docker compose up -d --build
```

Start existing services:

```
docker compose up -d
```

Check services:

```
docker compose ps
```

View all logs:

```
docker compose logs
```

View application logs:

```
docker compose logs codecruncher
```

View database logs:

```
docker compose logs database
```

Follow logs:

```
docker compose logs -f
```

Stop services:

```
docker compose stop
```

Remove containers and network:

```
docker compose down
```

Rebuild the application:

```
docker compose build
```

17. Docker Network

Docker Compose creates a dedicated network:

```
codecruncher-network
```

Network driver:

```
bridge
```

The containers attached to the network are:

```
codecruncher-app
  |
  |
  ▼
codecruncher-db
```

The Docker network allows containers to communicate without exposing the database directly to the host.

View Docker Networks

```
docker network ls
```

Inspect the CodeCruncher network:

```
docker network inspect codecruncher-network
```

The actual Compose-generated network name may include the Compose project name as a prefix.

18. PostgreSQL Database Container

The PostgreSQL service uses:

```
postgres:16-alpine
```

Container:

```
codecruncher-db
```

Database:

```
codecruncher
```

User:

```
codecruncher
```

Port:

```
5432
```

The database is intended to communicate through the internal Docker network rather than requiring public exposure of PostgreSQL.

PostgreSQL Health Verification

The database can be checked using:

```
docker exec codecruncher-db  
pg_isready -U codecruncher -d codecruncher
```

A successful result should indicate that PostgreSQL is accepting connections.

19. Docker Volume

PostgreSQL requires persistent storage so that database data does not disappear when the database container is recreated.

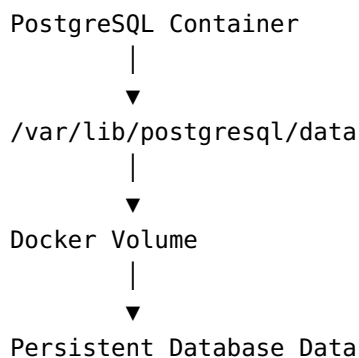
The Compose file defines:

```
codecruncher-db-data
```

The volume is mounted to:

```
/var/lib/postgresql/data
```

Architecture:



The Compose configuration explicitly defines the named volume for PostgreSQL persistence.

Docker Volume Commands

List volumes:

```
docker volume ls
```

Inspect a volume:

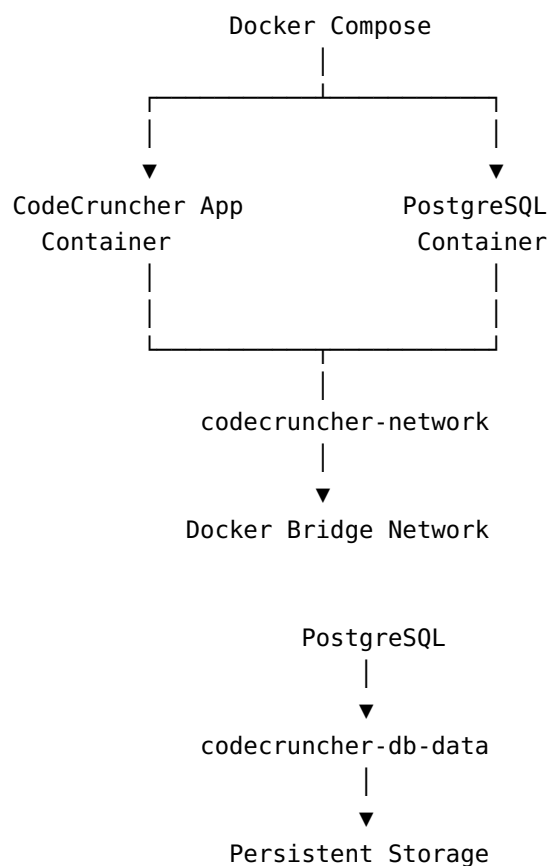
```
docker volume inspect codecruncher-db-data
```

Remove a volume:

```
docker volume rm codecruncher-db-data
```

Warning: Removing the database volume permanently removes the stored PostgreSQL data.

20. Complete Docker Compose Architecture



21. Complete Docker Workflow

The complete local Docker workflow is:

1. Clone the CodeCruncher repository
- ↓
2. Install Docker

- ↓
3. Verify Docker
 - ↓
 4. Review Dockerfile
 - ↓
 5. Build Docker image
 - ↓
 6. Verify Docker image
 - ↓
 7. Create Docker container
 - ↓
 8. Map port 8080 → 80
 - ↓
 9. Start Nginx
 - ↓
 10. Verify CodeCruncher application
 - ↓
 11. Stop/remove standalone container
 - ↓
 12. Start Docker Compose
 - ↓
 13. Create CodeCruncher container
 - ↓
 14. Create PostgreSQL container
 - ↓
 15. Create Docker network
 - ↓
 16. Create PostgreSQL volume
 - ↓
 17. Verify both services
 - ↓
 18. Verify PostgreSQL
 - ↓
 19. Test CodeCruncher application

22. Useful Docker Commands

Images

List images:

```
docker images
```

Build image:

```
docker build -t codecruncher:v1 .
```

Inspect image:

```
docker inspect codecruncher:v1
```

View image history:

```
docker history codecruncher:v1
```

Remove image:

```
docker rmi codecruncher:v1
```

Containers

List running containers:

```
docker ps
```

List all containers:

```
docker ps -a
```

Start:

```
docker start codecruncher-container
```

Stop:

```
docker stop codecruncher-container
```

Restart:

```
docker restart codecruncher-container
```

Logs:

```
docker logs codecruncher-container
```

Inspect:

```
docker inspect codecruncher-container
```

Remove:

```
docker rm codecruncher-container
```

Networks

List networks:

```
docker network ls
```

Inspect network:

```
docker network inspect codecruncher-network
```

Volumes

List volumes:

```
docker volume ls
```

Inspect volume:

```
docker volume inspect codecruncher-db-data
```

Compose

Start:

```
docker compose up -d
```

Build and start:

```
docker compose up -d --build
```

Check services:

```
docker compose ps
```

View logs:

```
docker compose logs
```

Stop:

```
docker compose stop
```

Remove:

```
docker compose down
```

23. Troubleshooting

Problem 1 — Container Name Already Exists

Error:

```
Conflict. The container name is already in use.
```

Check existing containers:

```
docker ps -a
```

Remove the existing container:

```
docker rm -f codecruncher-container
```

Start the container again:

```
docker run -d
  --name codecruncher-container
  -p 8080:80
  codecruncher:v1
```

Problem 2 — Port 8080 Already Allocated

Error:

```
Bind for 0.0.0.0:8080 failed:
port is already allocated
```

Find the container using the port:

```
docker ps
```

Stop it:

```
docker stop <container>
```

Remove it if required:

```
docker rm <container>
```

Alternatively, use another host port:

```
docker run -d
  --name codecruncher-container
  -p 8081:80
  codecruncher:v1
```

Problem 3 — Docker Build Fails

Check Docker:

```
docker info
```

Check files:

```
ls
```

Verify the Dockerfile:

```
cat Dockerfile
```

Build again:

```
docker build --no-cache -t codecruncher:v1 .
```

Problem 4 — Application Does Not Load

Check running containers:

```
docker ps
```

Check logs:

```
docker logs codecruncher-container
```

Check port mapping:

```
docker port codecruncher-container
```

Test locally:

```
curl -I http://localhost:8080
```

Problem 5 — Nginx Configuration Error

Inspect the container:

```
docker exec -it codecruncher-container sh
```

Test the Nginx configuration:

```
nginx -t
```

Exit:

```
exit
```

Problem 6 — PostgreSQL Is Not Ready

Check database logs:

```
docker compose logs database
```

Check database status:

```
docker exec codecruncher-db  
pg_isready -U codecruncher -d codecruncher
```

Check services:

```
docker compose ps
```

24. Docker Security Considerations

The following practices should be followed:

- Do not commit `.env` files containing secrets.
- Do not expose PostgreSQL port `5432` publicly unless required.
- Use environment variables for sensitive configuration.
- Avoid storing production passwords directly in Compose files.
- Keep Docker base images updated.
- Use trusted base images.
- Use minimal images such as Alpine where appropriate.
- Avoid running containers with unnecessary privileges.
- Use versioned image tags.
- Remove unused images and containers.
- Regularly scan images for vulnerabilities.
- Keep application dependencies updated.
- Restrict access to exposed ports.

The current Compose file contains a demonstration PostgreSQL password, so production deployments should replace demonstration credentials with secure secret management.

25. Docker Image Optimization

The project already follows several useful Docker optimization practices.

1. Alpine Base Images

The Dockerfile uses:

```
node:20-alpine
```

and:

```
nginx:1.27-alpine
```

Alpine images are lightweight and suitable for reducing unnecessary runtime components.

2. Multi-Stage Build

The Node.js build environment is separated from the Nginx runtime environment.

3. Production Runtime

Only the generated frontend files are copied into the Nginx image:

```
COPY --from=build /app/dist /usr/share/nginx/html
```

4. Reproducible Dependency Installation

The Dockerfile uses:

```
npm ci
```

which installs dependencies according to the lock file.

5. `.dockerignore`

The `.dockerignore` file prevents unnecessary files from being sent as part of the Docker build context.

26. Docker Best Practices

Recommended practices for the CodeCruncher project include:

Use Specific Image Tags

Instead of:

```
codecruncher:latest
```

prefer:

```
codecruncher:v1  
codecruncher:v2
```

Keep Runtime Images Minimal

Use only the components required to run the application.

Use Multi-Stage Builds

Separate application compilation from the production runtime.

Do Not Store Secrets in Images

Avoid placing passwords, API keys or tokens inside the Dockerfile.

Use Persistent Volumes

Database data should be stored in Docker volumes instead of the writable container layer.

Use Docker Networks

Keep application and database communication inside an isolated Docker network.

Monitor Logs

Use:

```
docker logs
```

and:

```
docker compose logs
```

for troubleshooting.

27. Docker Cleanup

List stopped containers:

```
docker ps -a
```

Remove stopped containers:

```
docker container prune
```

Remove unused images:

```
docker image prune
```

Remove unused networks:

```
docker network prune
```

Remove unused volumes:

```
docker volume prune
```

Remove unused Docker resources:

```
docker system prune
```

Warning: Cleanup commands can permanently remove unused Docker resources. Review the resources before confirming destructive cleanup.

28. Complete Verification Checklist

After deployment, verify the following:

Docker

```
docker --version
```

Docker Compose

```
docker compose version
```

Image

```
docker images
```

Container

```
docker ps
```

Application

```
curl -I http://localhost:8080
```

Logs

```
docker logs codecruncher-container
```

Compose Services

```
docker compose ps
```

Database

```
docker exec codecruncher-db  
pg_isready -U codecruncher -d codecruncher
```

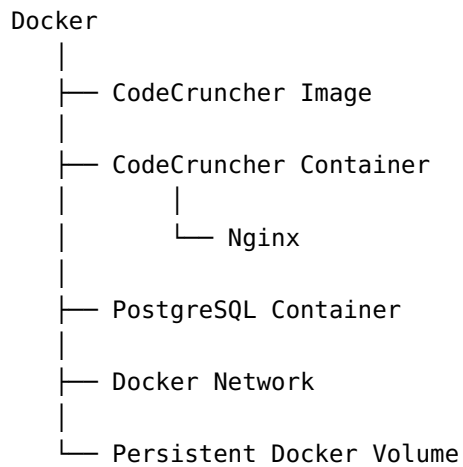
Network

```
docker network ls
```

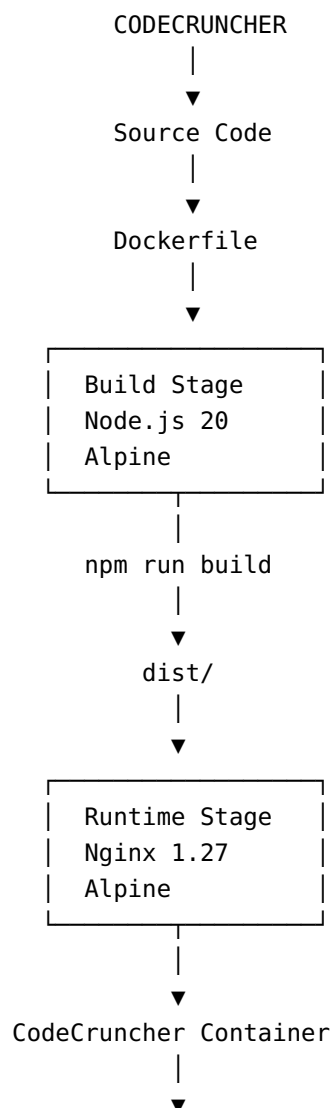
Volume

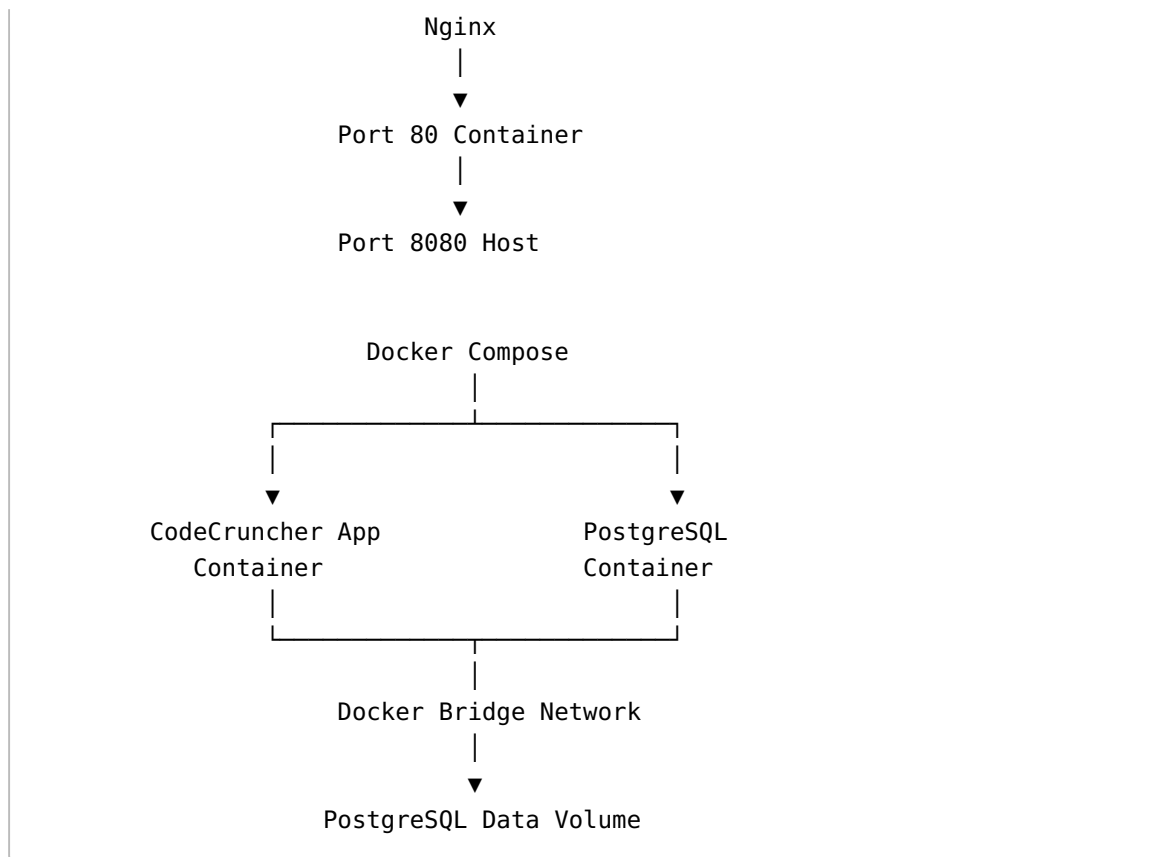
```
docker volume ls
```

A successful deployment should show:



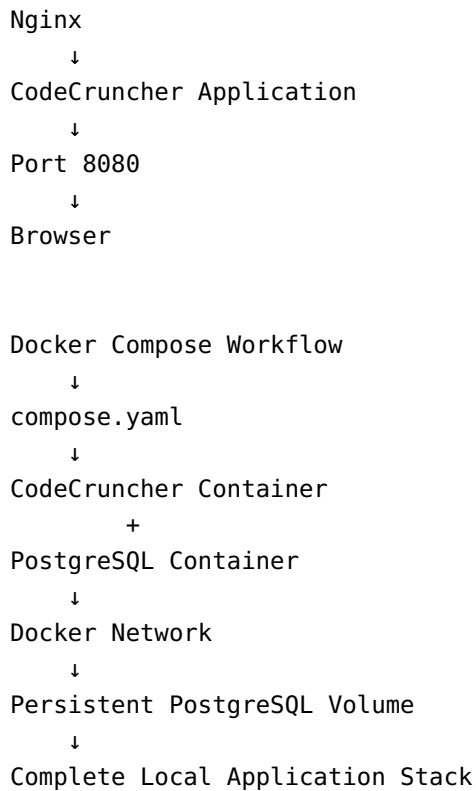
29. Final Docker Architecture





30. Complete Project Workflow

```
Developer
↓
CodeCruncher Source Code
↓
Dockerfile
↓
Multi-Stage Docker Build
↓
Node.js Build Stage
↓
npm ci
↓
npm run build
↓
dist/
↓
Nginx Runtime Image
↓
Docker Image
↓
Docker Container
↓
```



31. Project Result

The CodeCruncher application was successfully structured for Docker-based containerization.

The Docker implementation demonstrates:

- Docker installation and verification.
- Docker image creation.
- Multi-stage Docker builds.
- Node.js-based frontend compilation.
- Nginx-based production serving.
- Docker container execution.
- Port mapping.
- Container logging.
- Container inspection.
- Docker Compose.
- PostgreSQL containerization.
- Docker networking.
- Persistent Docker volumes.
- Application verification.
- Docker troubleshooting.
- Docker security practices.
- Docker image optimization.

The Docker Compose architecture allows the CodeCruncher application and PostgreSQL database to run together as an isolated multi-container application stack.

Conclusion

This project provides practical experience with Docker containerization of a modern React, TypeScript and Vite web application.

The CodeCruncher Dockerfile demonstrates a production-oriented multi-stage build in which Node.js is used to compile the frontend and Nginx is used as the lightweight runtime server.

Docker Compose extends the project into a multi-container architecture by running the CodeCruncher application together with PostgreSQL. The services communicate through a dedicated Docker bridge network, while PostgreSQL uses a persistent Docker volume for database storage.

The project therefore demonstrates the complete Docker lifecycle:

Source Code → Dockerfile → Docker Image → Docker Container → Nginx → Application

and the multi-container workflow:

Docker Compose → Application Container + PostgreSQL Container → Docker Network → Persistent Volume

This project establishes a strong practical foundation in Docker, containerization, Docker Compose, Nginx, networking, persistent storage, troubleshooting and container security.